NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

ATTRIBUTE GRAMMARS

IN OFLAT

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *October 29, 2024*

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ATTRIBUTE GRAMMARS

IN OFLAT

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

Adviser: Artur Miguel Dias
Professor Auxiliar, DI-FCT, Universidade NOVA de Lisboa

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *October 29, 2024*

Abstract

Theoretical concepts in Formal Languages and Automata Theory (FLAT) are known for their formal, rigorous, and often complex nature, making them challenging to learn. To address this challenge, various tools have been developed over the years, allowing users to engage with these concepts through practical exercises and examples.

At FCT-UNL, two pedagogical tools were created to aid in this learning process: the OCaml-FLAT library, which provides logical support for FLAT concepts, and the OFLAT graphical application, a web-based tool offering interactive visualizations of these ideas.

The first goal of this dissertation is to extend the OCaml-FLAT library by incorporating support for attribute grammars. The second goal is to enhance the OFLAT graphical application by adding features that allow for the visualization and interaction with these grammar concepts.

A key focus of this work is the pedagogical enhancement of the developed software. Consequently, a significant portion of the dissertation will be devoted to explaining the theoretical foundations of attribute grammars, including various classic variants.

Keywords: Formal Languages, Automata Theory, OCaml, OFLAT, OCaml-FLAT, Functional Programming, Formal Grammars, Pedagogical Tools

Resumo

Os conceitos teóricos em Teoria de Linguagens Formais e Autómatos (FLAT) são conhecidos pela sua natureza formal, rigorosa e muitas vezes complexa, o que os torna desafiantes para aprender. Para mitigar essa dificuldade, várias ferramentas foram desenvolvidas ao longo dos anos, permitindo que os utilizadores interajam com esses conceitos através de exercícios e exemplos práticos.

Na FCT-UNL, foram criadas duas ferramentas pedagógicas com este propósito: a biblioteca OCaml-FLAT, que fornece suporte lógico para os conceitos de FLAT, e a aplicação gráfica OFLAT, uma ferramenta web que oferece visualizações interativas desses conceitos.

O primeiro objetivo desta dissertação é estender a biblioteca OCaml-FLAT, incorporando suporte para gramáticas com atributos. O segundo objetivo é melhorar a aplicação gráfica OFLAT, adicionando funcionalidades que permitam a visualização e interatividade com estes conceitos de gramáticas.

Um foco importante deste trabalho é o reforço do caráter pedagógico do software a ser desenvolvido. Assim, uma parte significativa da dissertação será dedicada à explicação dos fundamentos teóricos das gramáticas com atributos, incluindo as suas diferentes variantes clássicas.

Palavras-chave: Teoria de Linguagens Formais, Autómatos, OCaml, OFLAT, OCaml-FLAT, Programação Funcional, Gramáticas Formais, Ferramentas Pedagógicas

Contents

List of Figures **v**

1 Introduction **1**
1.1 Context 1
1.2 Objectives 1
1.3 Document Structure 1

2 Theory of Computation **2**
2.1 Chomsky Hierarchy 2
2.2 Context-free Languages 2
2.2.1 With a Local LaTeX Installation 3

3 A Short LaTeX Tutorial with Examples **4**
3.1 Document Structure 4
3.2 Dealing with Bibliography 4
3.3 Inserting Tables 4
3.4 Importing Images 4
3.5 Floats, Figures and Captions 4
3.6 Text Formatting 6
3.7 Generating PDFs from LaTeX 6
3.7.1 Generating PDFs with pdflatex 6
3.7.2 Dealing with Images 7
3.7.3 Dealing with Citations 7
3.7.4 Footnotes 7
3.7.5 Tables 7
3.7.6 Figures 8
3.8 Equations 8
3.9 Test for algorithms 8

4 OCaml-Flat and OFLAT Systems **12**

4.1 OCamlFLAT 12
4.2 OFLAT Platform 13

5 Technology **14**
5.1 OCaml/JavaScript Interoperability 14
5.1.1 JavaScript Types 14
5.1.2 Bindings 15
5.1.3 `js_of_ocaml` Modules 15
5.2 Cytoscape 15

6 Lorem Ipsum **16**

Bibliography **17**

List of Figures

3.1 A figure with two sub-figures! 5
3.2 Bitmap image (JPG/PNG) 9
3.3 Vectorial image (PDF) 10
3.4 Exemplo de utilização de *subbottom* 11

1

Introduction

1.1 Context

The study of Formal Languages and Automata Theory (FLAT) is integral to Computer Engineering and Computer Science programs. However, due to its abstract and mathematical nature, students often struggle to fully grasp the concepts and models involved. Therefore, a tool that complements the study of FLAT theory by being practical, user-friendly, and visually oriented is highly beneficial.

In recent years, the OCaml-FLAT/OFLAT tool has been developed at FCT-UNL, offering extensive support in this field. Despite its advanced state, with support for several key models, there are still many features that could enhance its functionality.

This dissertation aims to introduce new capabilities to OCamlFLAT/OFLAT, enhancing both the OCamlFLAT library and the OFLAT graphical interface.

1.2 Objectives

The primary objective of this dissertation is to incorporate support for attribute grammars. Attribute grammars extend context-free grammars by allowing additional information (attributes) to be associated with production rules. These attributes are crucial for describing the semantic properties of the languages generated by the grammars.

To achieve this, the OCamlFLAT library needs to be extended to support attribute grammars. Additionally, graphical enhancements are required to enable the creation and editing of these grammars within the OFLAT platform.

1.3 Document Structure

2

Theory of Computation

In the Theory of Computation, various models of computation with different levels of expressive power are studied. To discuss and compare different levels of expressive power, it is inevitable to consider algorithms in the context of some application domain. The typically chosen application domain is the syntax of formal languages. There are two techniques for defining languages that are typically used. The first technique is based on the concept of generation and uses the formalism of grammars. Another technique is based on the mechanism of recognition and uses the formalism of automata.

2.1 Chomsky Hierarchy

Noam Chomsky (with the help of Marcel-Paul Schützenberger) identified several classes of formal languages, with different levels of specification complexity, requiring computational models with different levels of expressive power to solve the problem of recognizing these languages.

There are four levels: type 0, type 1, type 2, and type 3.

- Type 0 — Recursively enumerable languages
- Type 1 — Context-sensitive languages
- Type 2 — Context-free languages
- Type 3 — Regular languages

The Chomsky hierarchy involves four levels of language types. Each type of language is associated with a type of generator and a type of recognizer.

2.2 Context-free Languages

Context-free languages include regular languages and are a type of formal language defined by context-free grammars. These grammars have production rules that generate sequences of symbols recursively, using non-terminal and terminal symbols. They

have various applications in computer science, such as aiding in the implementation of compilers and describing document formats

2.2.1 With a Local LaTeX Installation

3

A Short LaTeX Tutorial with Examples

This Chapter aims at exemplifying how to do common stuff with LaTeX. We also show some stuff which is not that common! ;)

Please, use these examples as a starting point, but you should always consider using the *Big Oracle* (aka, Google, your best friend) to search for additional information or al-ternative ways for achieving similar results.

3.1 Document Structure

3.2 Dealing with Bibliography

Citing something online [4, 6, 8].

3.3 Inserting Tables

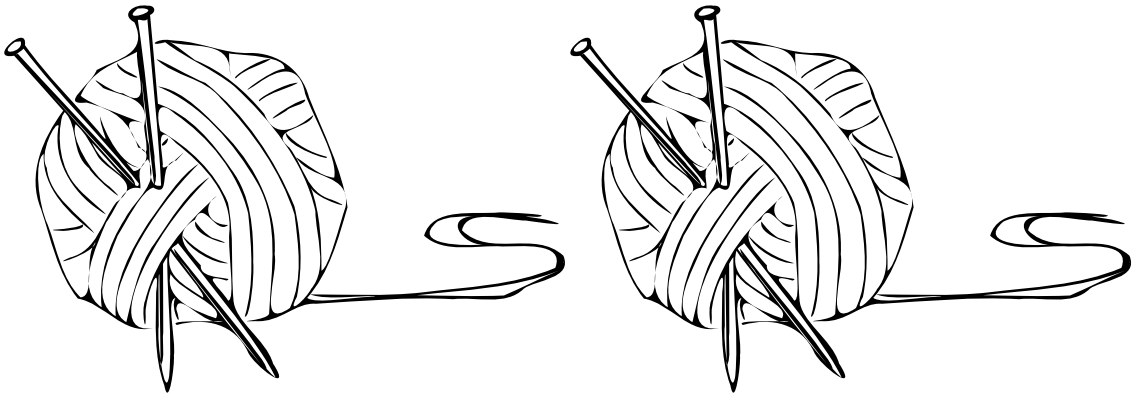
3.4 Importing Images

3.5 Floats, Figures and Captions

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.



(a) One sub-figure (b) Another sub-figure

Figure 3.1: A figure with two sub-figures!

And this is a small text that references the Figure 3.1 and its Subfigures 3.1(a) and 3.1(b).

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan

eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

3.6 Text Formatting

3.7 Generating PDFs from LaTeX

3.7.1 Generating PDFs with pdflatex

You may create PDF files either by using `latex` to generate a DVI file, and then use one of the many DVI-2-PDF converters, such as `dvipdfm`.

Alternatively, you may use `pdflatex`, which will immediately generate a PDF with no intermediate DVI or PS files. In some systems, such as Apple, PDF is already the default format for LaTeX. I strongly recommend you to use this approach, unless you have a very good argument to go for `latex` + `dvipdfm`.

A typical pass for a document with figures, cross-references and a bibliography would be:

```
$ pdflatex template
$ bibtex template
$ pdflatex template
$ pdflatex template
```

You will notice that there is a new PDF file in the working directory called `template.pdf`. Simple :)

Please note that, to be sure all table of contents, cross-references and bibliographic citations are up-to-date, you must run `latex` once, then `bibtex`, and then `latex` twice.

3.7.2 Dealing with Images

You may process the same source files with both `latex` or `pdflatex`. But, if your text include images, you must be careful. `latex` and `pdflatex` accept images in different (exclusive) formats. For `latex` you may use EPS ou PS figures. For `pdflatex` you may use JPG, PNG or PDF figures. I strongly recommend you to use PDF figures in vectorial format (do not use bitmap images unless you have no other choice).

3.7.3 Dealing with Citations

Para fazer citações, deverá usar-se a chave da referência no ficheiro BibTeX. Se for uma única referência [2], usar um "~" para ligar o `\cite{...}` à palavra que o precede (...`referência~\cite{Artho04}`). Caso queira fazer múltiplas citações [10, 11, 5], deverá agrupá-las dentro de um úinico `\cite{...}`.

Note que o ficheiro de bibliografia pode ter tantas entradas quantas quiser. Apenas aquelas cuja chave seja referenciada no texto é que serão incluidas na listagem de bibliografia.

3.7.4 Footnotes

Footnotes[1] will be numbered and shown in the bottom of the page.

3.7.5 Tables

The Table 3.1 illustrates some important concepts associated with table construction:

i) Do not use vertical lines;

ii) The caption should be above the table;

iii) Use the macros `\toprule`, `\midrule` and `\bottomrule` to make the top, inner and bottom horizontal lines, respectively.

Table 3.1: Test results summary.

Test	Anomalies	Warnings	Correct	Categories	Missed
Connection [3]	2	2	1	*C*	1
Coordinates'03 [1]	1	4	1	*2B, 1C*	0
Local Variable [1]	1	2	1	*A*	0
NASA [1]	1	1	1	—	0
Coordinates'04 [2]	1	4	1	*3C*	0
Buffer [2]	0	7	0	*2A, 1B, 2C, 2D*	0
Double-Check [2]	0	2	0	*1A, 1B*	0
StringBuffer [7]	1	0	0	—	1
Account [12]	1	1	1	—	0

[1] This is a simple footnote.

Jigsaw [12]	1	2	1	*C*	0
Over-reporting [12]	0	2	0	*1A, 1C*	0
Under-reporting [12]	1	1	1	—	0
Allocate Vector [9]	1	2	1	*C*	0
Knight Moves [3]	1	3	1	*2B*	0
Total	12	33	10	5A, 6B, 10C, 2D	2

3.7.6 Figures

The images inserted in the document must be of good quality, preferably in vector format (vector PDF) and not in *bitmap* (PNG, JPG, etc.). *bitmap* images (Figure 3.2) do not scale well and have negative effects on the quality of your document. On the other hand, *vector* images Figure 3.3 scale as much as necessary without degrading the quality of the image.

You should only use *screenshots* for your plots, charts, etc, if you absolutely have no other alternative. Instead of generating a *screenshot*, try using a virtual PDF printer and printing to a PDF file. As a general rule, you will get a vector PDF. Even if your PDF contains images, they will always be of higher or equal quality than what you would get with a *screenshot*.

To combine several figures into a single one... You can then reference the set as Figure 3.4 or the sub-figures separately as 3.7.6 and 3.7.6.

3.8 Equations

LaTeX is a powerful tool for writing in a mathematical style. It allows you to insert formulas into the text, such as this: $ax^2 + bx + c = 0$. It also allows formulas to be highlighted on a separate line and centered on the page.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

or numbererd

$$e = mc^2 \tag{3.1}$$

which can latter be referenced as equation 3.1

3.9 Test for algorithms

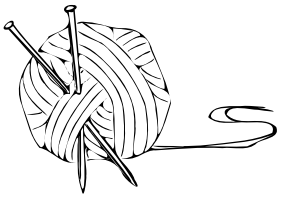
Uncomment the algorithms source below and add the following to file "`5_packages.tex`"

```
\usepackage{algorithm2e}
\RestyleAlgo{ruled}
```

and uncomment

Figure 3.2: Bitmap image (JPG/PNG)

Figure 3.3: Vectorial image (PDF)



(a) Novelo de lã

(b) Tempestade com neve

Figure 3.4: Exemplo de utilização de *subbottom*

```
\ntaddlistof{listofalgorithms}
```

in file "`8_list_og.tex`".

4

OCaml-Flat and OFLAT Systems

The OCamlFLAT library and the OFLAT platform are the main componentsof this thesis, both developed in OCaml. They are already in an advanced stage of development, thanks to the work of several students and professorswho have contributed to this project.

The OCamlFLAT library consists of various modules that make up the logical part of the system, where several FLAT theory models have already been implemented. The OFLAT platform, on the other hand, is a web application that provides the system's graphical interface.

4.1 OCamlFLAT

The OCamlFLAT library is organized into modules to ensure a clearer and more structured organization. There are modules that implement models of FLAT theory, such as regular expressions, finite automata, and context-free grammars, as well as auxiliary modules for tests, error handling, and parsers. It is important to note that the modules follow a hierarchical relationship among themselves, which facilitates the maintenance and expansion of the library.

Some of the most important modules are:

Entity: This module represents the most abstract level of the library. The entities are the exercises and the FLAT models, each containing a type, a description, and a unique ID.

Exercise: This module is responsible for supporting exercises, which are validated using UnitTests. It extends the Entity module, inheriting its properties and methods.

Model: An abstract module that also extends Entity and introduces common functionalities to the FLAT models available in the library. Examples of methods included in this module are validate, which checks the validity of a model, and accept, which processes inputs according to the model.

Examples of modules that represent FLAT models and extend Model include:

RegularExpression: This module contains all the functionalities needed to work with regular expressions, allowing the definition and manipulation of these expressions.

FiniteAutomaton: This module implements the functionalities of a finite automaton, including the definition of states and transitions.

ContextFreeGrammar: This module offers the functionalities of a context-free grammar, allowing the definition of production rules and the analysis of strings.

PushdownAutomaton: This module contains the functionalities of a pushdown automaton, which is an extension of finite automata with an additional stack for temporary storage of information.

These modules are designed to work together, forming a robust and flexible library for the implementation and analysis of FLAT theory models.

4.2 OFLAT Platform

The OFLAT platform is the graphical interface of the project, available online to the public. Its main goal is to serve as an easy-to-use educational tool. It features a static menu on the left sidebar and a dynamic workspace on the right, which changes according to the options selected on the screen. In Figure 4.1, you can see the main screen when you click on the "New model -> Finite automata" option in the sidebar.

The platform is intuitive and user-friendly. For example, to create a finite automaton, you first need to choose the option in the sidebar and then right-click on the blank workspace, where the option to create a state appears. The state can be either final or initial (Figure 4.2), and an initial state is already displayed by default. By right-clicking on the state, you can add transitions (Figure 4.3) and thus build a finite automaton. In the lower-left corner of the white screen, information about the automaton is displayed, such as whether it is deterministic, whether it is minimal, the number of useless states, the number of states, and transitions. In Figure 4.4, we can see that the created automaton is deterministic and not minimal. We can then click on the "minimize automaton" option to view a minimal version (Figure 4.5).

5

Technology

5.1 OCaml/JavaScript Interoperability

Interoperability between OCaml and JavaScript is made possible through the use of the `js_of_ocaml` toll, which translates OCaml code into JavaScript, allowing OCaml code to run in a browser.

This tool consists of three main components:

- `js_of_ocaml` ppx: A preprocessor that extends the OCaml library, adding functionalities that facilitate integration with JavaScript.
- `js_of_ocaml` library: A library that contains bindings for various categories of JavaScript objects, enabling OCaml code to efficiently interact with JavaScript APIs and libraries.
- `js_of_ocaml` compiler: A compiler that translates OCaml code into JavaScript, allowing OCaml programs to be executed in web-based environments.

5.1.1 JavaScript Types

Types in OCaml and JavaScript are not represented in the same way in both languages. For example, a string in OCaml is a mutable array of bytes, whereas in JavaScript, strings are constant arrays of UTF-16 code points. Therefore, it is necessary to find a way to represent JavaScript types in OCaml appropriately. The table below illustrates the correspondences between the types in the two languages.

To ensure effective interoperability, it is crucial to understand how each type is handled in OCaml and JavaScript. For instance, besides strings, other types such as numbers, arrays, and objects also have distinct representations. In OCaml, integers and floating-point numbers are treated differently compared to JavaScript, where all numbers are essentially floating-point. Arrays in OCaml are mutable, while in JavaScript, arrays can be either mutable or immutable, depending on the context.

The table below summarizes the type correspondences between OCaml and JavaScript:

OCaml	JavaScript in OCaml	JavaScript
int	int	Number
float	float	Number
bool	bool Js.t	Boolean
string	`Js.js_string Js.t`	String
array	`Js.js_array Js.t`	Array

5.1.2 Bindings

5.1.3 `js_of_ocaml` Modules

The `js_of_ocaml` library already includes several modules with predefined bindings, but only a few are necessary for the development of the OFLAT platform.

Below are some examples of these modules and submodules:

- Js Module: This module contains definitions of JavaScript types, functions for standard JavaScript objects, and conversion functions between OCaml and JavaScript types. It is essential for ensuring interoperability between the two languages.
- Js.Unsafe Submodule: This submodule allows for operations on JavaScript that are not safe, offering greater flexibility but requiring extra caution to avoid errors and vulnerabilities.
- `Dom_html` Module: Defines and manipulates HTML DOM elements, enabling the creation and modification of HTML structures directly from OCaml code. This module is crucial for building the graphical interface of the platform.
- Firebug Module: Allows for the visualization and debugging of objects in the browser's developer console, facilitating the development and debugging process.

5.2 Cytoscape

Cytoscape is an open-source software platform that, although initially designed for biological research, is now a general tool for graph visualization and analysis. In this project, the OFLAT platform is used to visualize the various models supported by the application. To use this library, it is necessary to create bindings for OCaml/JavaScript interoperability.

6

Lorem Ipsum

Bibliography

[1] C. Artho, K. Havelund, and A. Biere. *High-Level Data Races*. 2003. URL: citeseer.ist.psu.edu/artho03highlevel.html (cit. on p. 7).

[2] C. Artho, K. Havelund, and A. Biere. "Using Block-Local Atomicity to Detect Stale-Value Concurrency Errors". In: *ATVA*. Ed. by F. Wang. Vol. 3299. Lecture Notes in Computer Science. Springer, 2004, pp. 150–164. ISBN: 3-540-23610-4 (cit. on p. 7).

[3] N. E. Beckman, K. Bierhoff, and J. Aldrich. "Verifying Correct Usage of Atomic Blocks and Typestate". In: *SIGPLAN Not.* 43.10 (2008), pp. 227–244. ISSN: 0362-1340. DOI: http://doi.acm.org/10.1145/1449955.1449783 (cit. on pp. 7, 8).

[4] W. contributors. *Shunting-yard algorithm — Wikipedia, The Free Encyclopedia*. 2017-03. URL: https://en.wikipedia.org/w/index.php?title=Shunting-yard_algorithm&oldid=817901155 (visited on 2018-03-01) (cit. on p. 4).

[5] J. Eliot and B. Moss. *Nested transactions: an approach to reliable distributed computing*. Cambridge, MA, USA: Massachusetts Institute of Technology, 1985. ISBN: 0-262-13200-1 (cit. on p. 7).

[6] *Fast Lexical Analyser*. URL: https://github.com/westes/flex (visited on 2020-07-26) (cit. on p. 4).

[7] C. Flanagan and S. N. Freund. "Atomizer: a dynamic atomicity checker for multi-threaded programs". In: *POPL '04: Proceedings of the 31st ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. Venice, Italy: ACM, 2004, pp. 256–267. ISBN: 1-58113-729-X. DOI: http://doi.acm.org/10.1145/964001.964023 (cit. on p. 7).

[8] *Gnu Bison*. URL: https://www.gnu.org/software/bison/ (visited on 2020-07-26) (cit. on p. 4).

[9] *IBM's Concurrency Testing Repository* (cit. on p. 8).

[10] N. Shavit and D. Touitou. "Software transactional memory". In: *PODC '95: Proceedings of the fourteenth annual ACM symposium on Principles of distributed computing*. Ottowa, Ontario, Canada: ACM, 1995, pp. 204–213. ISBN: 0-89791-710-3. DOI: http://doi.acm.org/10.1145/224964.224987 (cit. on p. 7).

[11] A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts*. Fifth. McGraw-Hill, 2006. ISBN: 007-124476-X (cit. on p. 7).

[12] C. von Praun and T. R. Gross. "Static Detection of Atomicity Violations in Object-Oriented Programs". In: *Journal of Object Technology*. 2003, p. 2004 (cit. on pp. 7, 8).